SC24 Atlanta, GA | hpc creates.

Versatile Datapath Soft Error Detection on the Cheap for HPC Applications

Yafan Huang, Sheng Di, Zhaorui Zhang, Xiaoyi Lu, Guanpeng Li

The University of Iowa — Organized Feb. 25th 1847

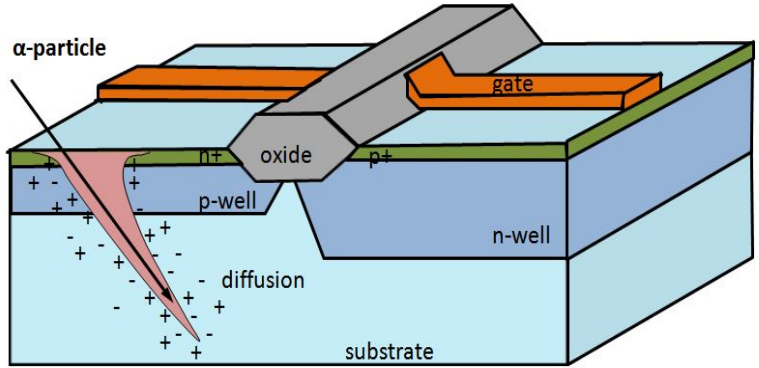
Argonne National Laboratory

The Hong Kong Polytechnic University 香港理工大學

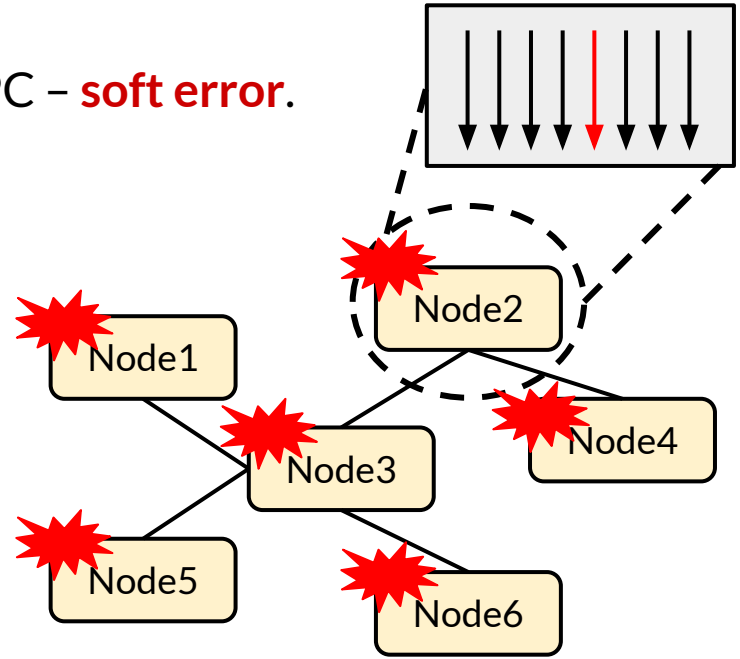
The University of California Merced — Let There Be Light — 2005

Soft Error

- An old, notorious, but frequent threat to HPC – **soft error**.



Soft Error Phenomenon[1]



00011000 ⇨ 00111000

Corrupted Logic Values in Hardware

Once a soft error occurs, it may propagate everywhere in HPC environments!

[1] MPS Lab @ Arizona State University, https://labs.engineering.asu.edu/mps-lab/research/error-resilience/

Failure Outcomes of Soft Error

- **Benign**: Error is masked somewhere and program *finishes normally*.
- **Crash/Hang**: Program crashes/hangs, and *exits abnormally*.
- **Silent Data Corruption (SDC)**: Program *finishes normally but gives wrong result* without notifying users – the most insidious failure outcome!

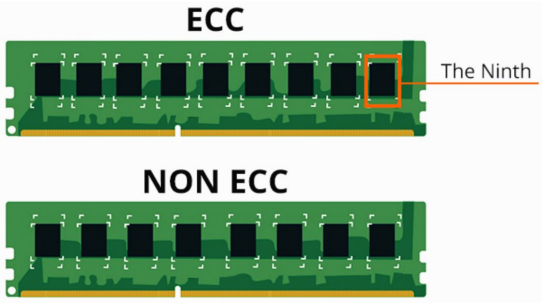
aws [1] Meta [2] Google [3] Alibaba.com™ [4]

Soft error is an imminent threat to HPC and hence should be mitigated!

[1] Amazon S3 Incident: https://aws.amazon.com/message/41926/
[2] [TechReport@Meta] Silent data corruption at scale
[3] [HotOS'21] Cores that don't count
[4] [SOSP'24] Understanding Silent Data Corruptions in a Large Production CPU Population

Mitigation Techniques

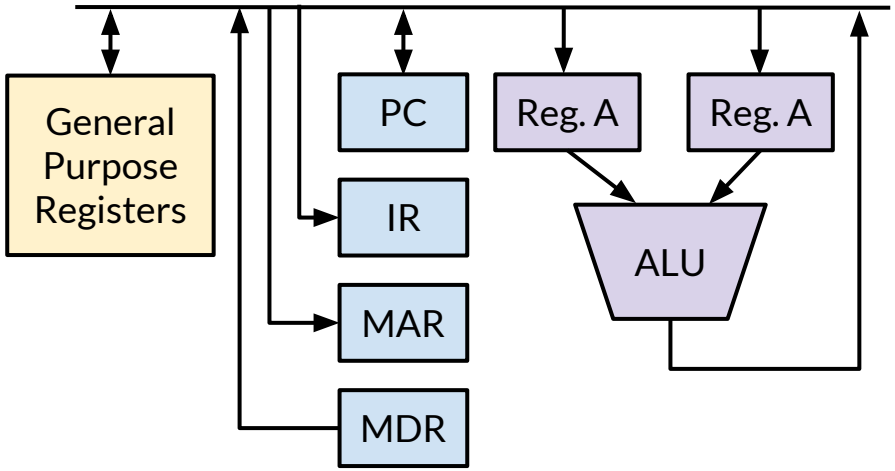
- **Memory errors** can be effectively protected by *Error Correction Code (ECC)*.
- **Datapath errors** are much more expensive to detect using hardware solutions.



EVEN PARITY

Byte		Parity Bit		
00000000	+	0	= 0	✓
10000001	+	1	= 3	⊘
11100000	+	1	= 4	✓
10000000	+	1	= 2	✓

ECC and its Principle[1]



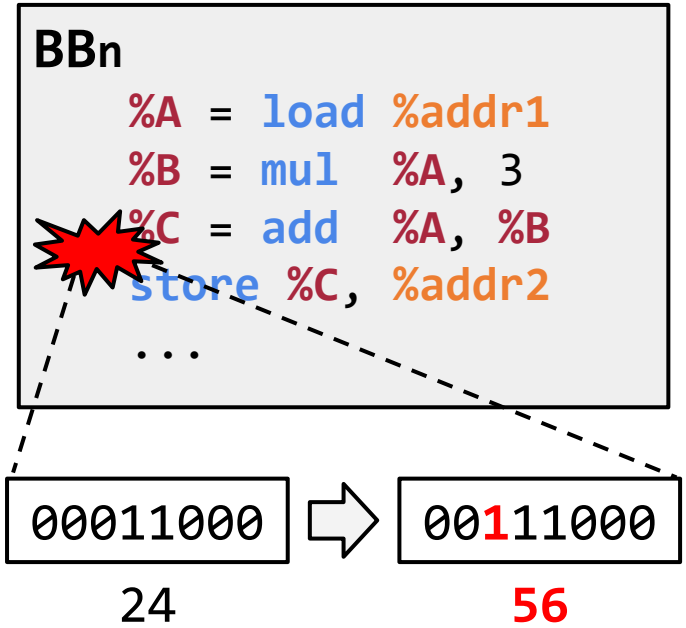
A simple one-bus organization datapath

Detecting datapath errors with software solutions!

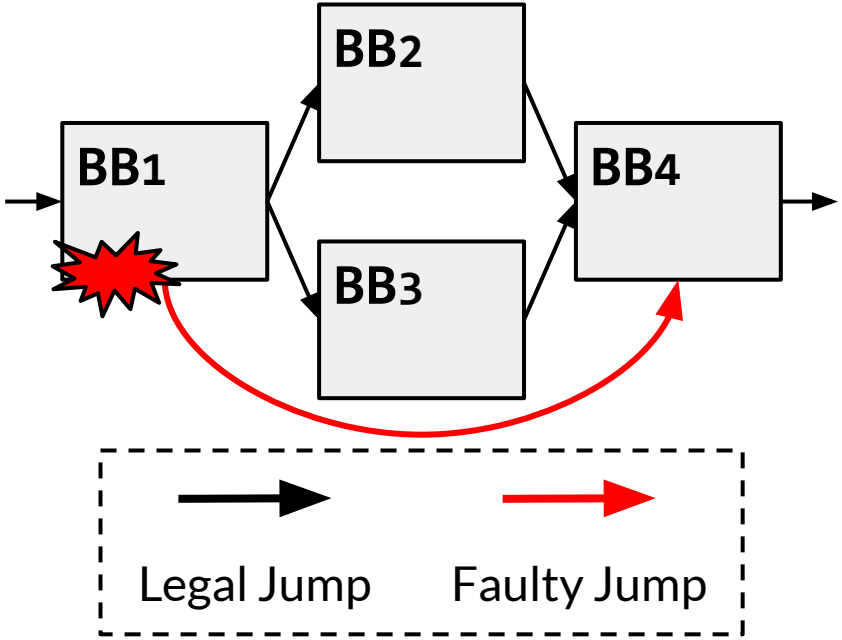
[1] https://community.fs.com/article/ecc-vs-non-ecc-memory-which-one-is-better.html

Activating Datapath Errors

- Once error *occur* in datapath, it *activate* at program-level in **two distinct patterns**.



Data-flow error



Control-flow error

Existing Works and Limitations

[EDDI, ToR'02],
[SDCTune, TECS'17],
[SInRG, SC'18],
[FT-CNN, TPDS'20],
[MinpSID, SC'22], etc.

Pros: *Effective Protection*

Cons: **Only Data-flow**

[CFCSS, ToR'02],
[YACCA, DFT'03],
[CDEA, ToC'11],
[ACS, LCTES'13],
[PaSS, LCTES'20], etc.

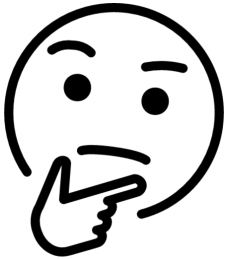
Pros: *Effective Protection*,

Cons: **Only Control-flow**

[SWIFT, CGO'05],
[gZDC, TDSC'23], etc.

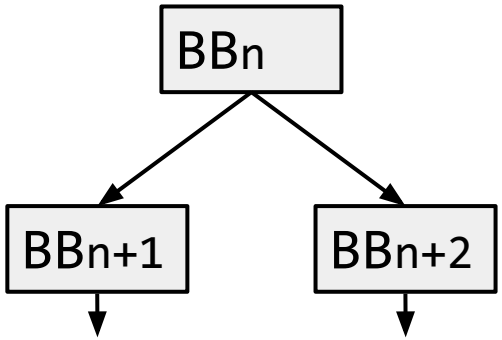
Pros: *Entire Datapath*,

Cons: **High Runtime O.H.**

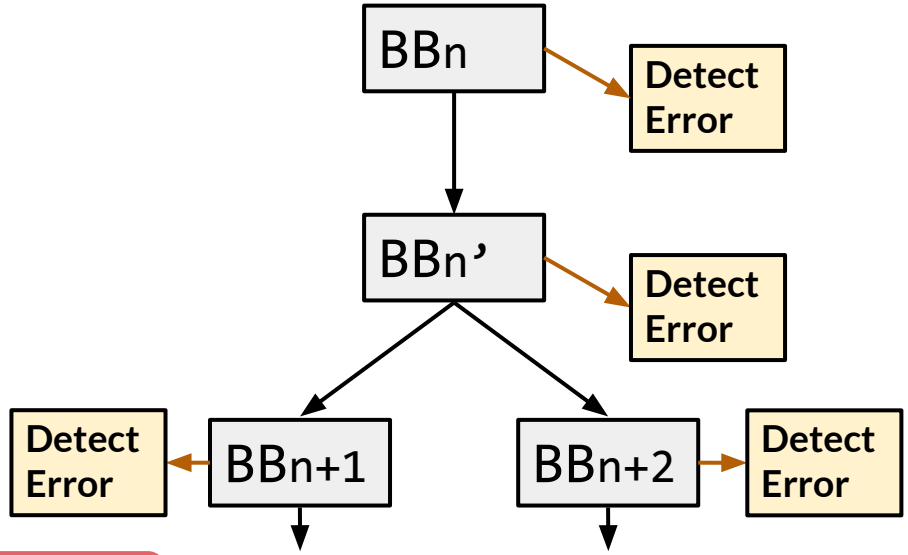


Where does that high runtime overhead come from?

Defect 1: Complicated Program Control-flow



Original program code



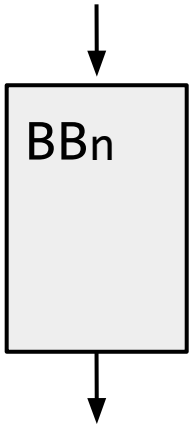
Code with protection

In EDDI protection, >200% extra executed BBs!

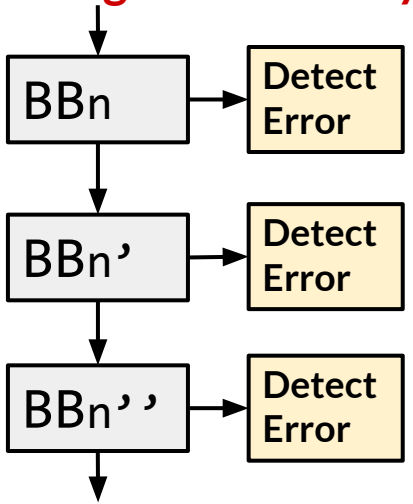
- **One basic block** in original code may be **splited into multiple ones** due to checking.
- **Error-detection branches itself** also complicates program control-flow.

Defect 2: Frequent Memory Accesses

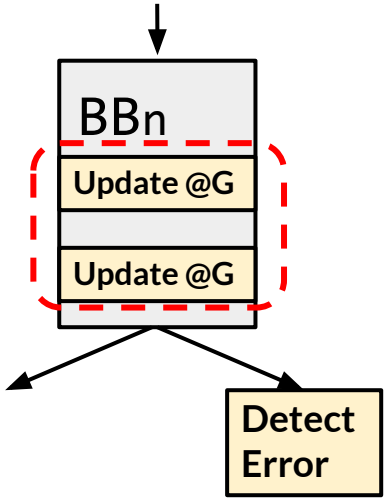
- To avoid such complicated program control-flow, some protection techniques **buffer** error-checking **results into global memory**.



Original program code



Protected code
(with complicated CFG)



Protected code
(with simplified CFG)

Extra global memory access required, reducing program runtime!

Goal and Requirements

- *Goal*: Providing robust **entire datapath protection** with **low runtime overhead**.

Protection

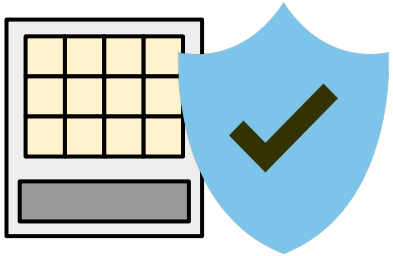
1. Data-flow protection
2. Control-flow Protection

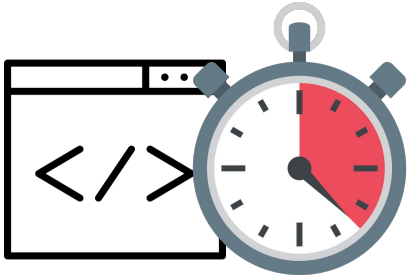
Overhead

1. Maintain program CFG
2. Avoid complex memory access patterns.

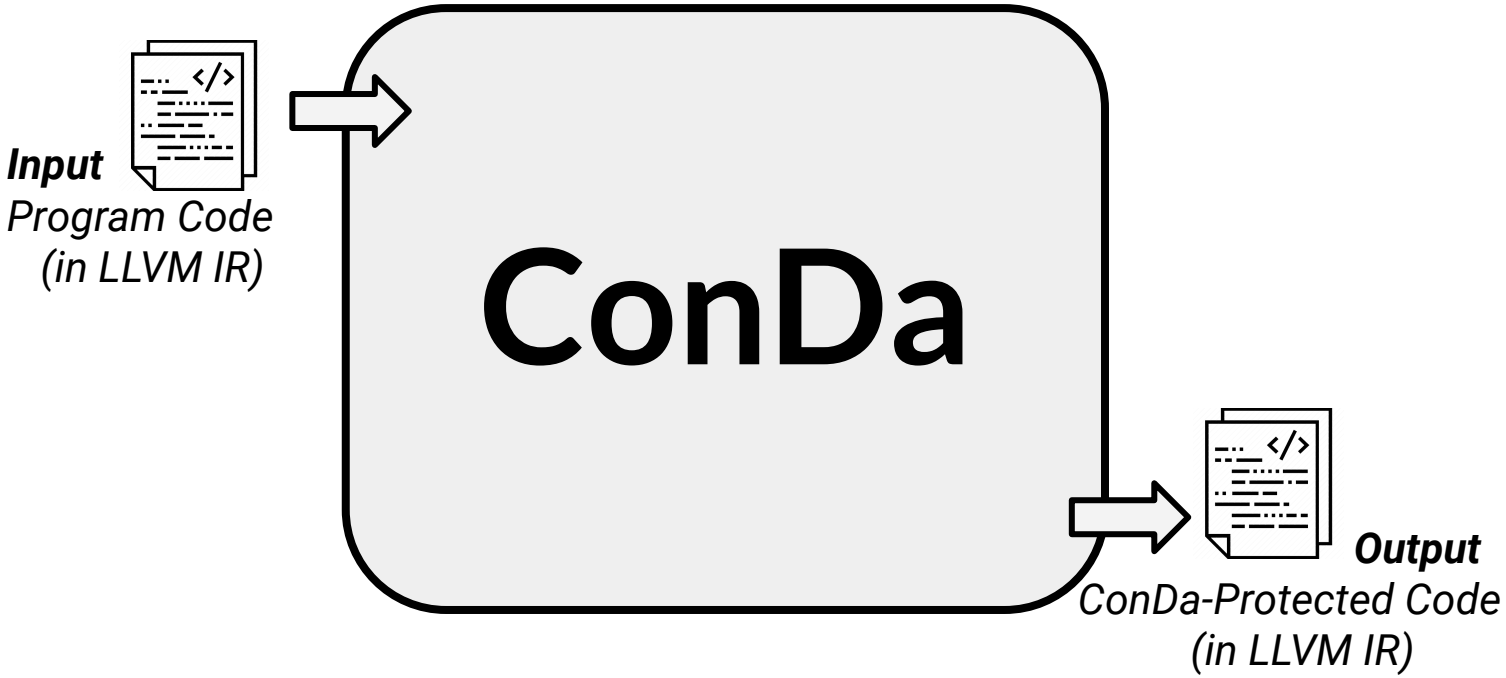
Usability

1. Generic & Plug-and-play
2. No expensive dynamic profiling.



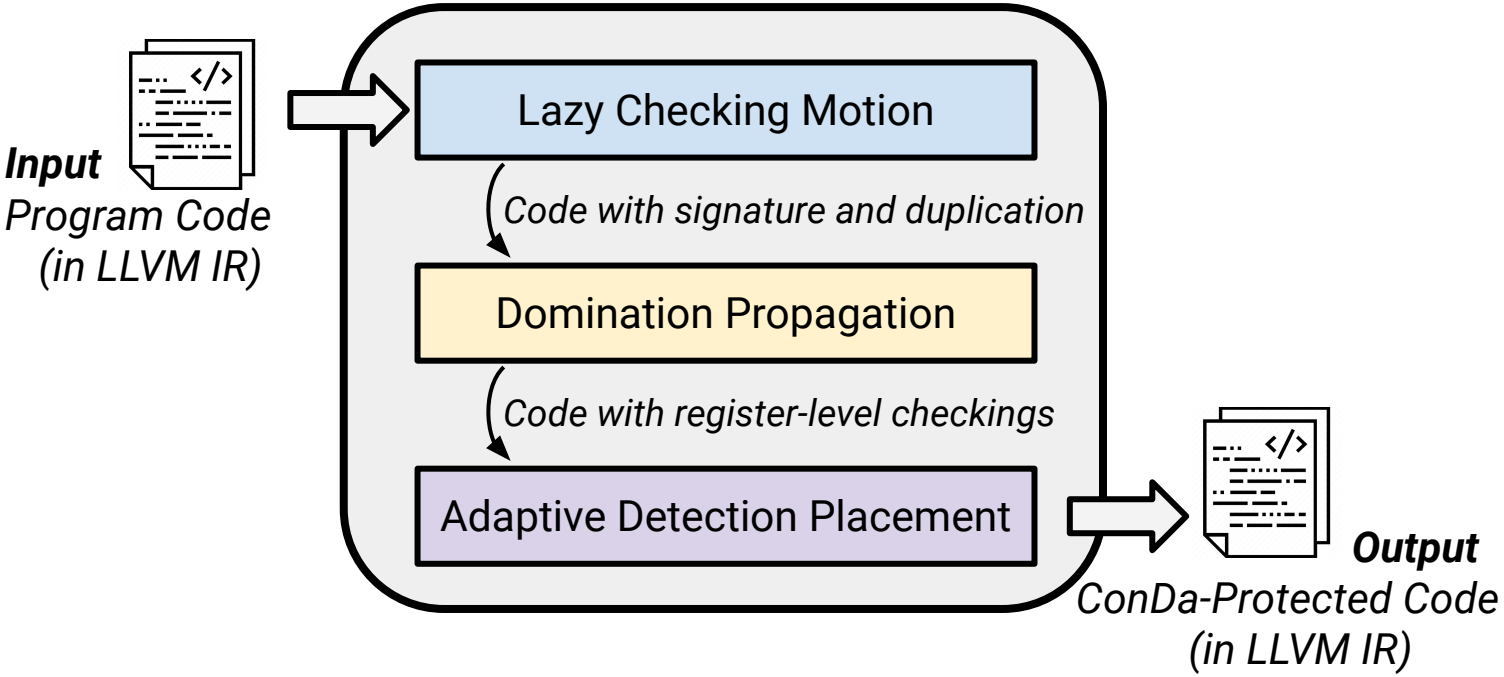


Our Solution: ConDa



One static code transformation without dynamic profiling.

Our Solution: ConDa



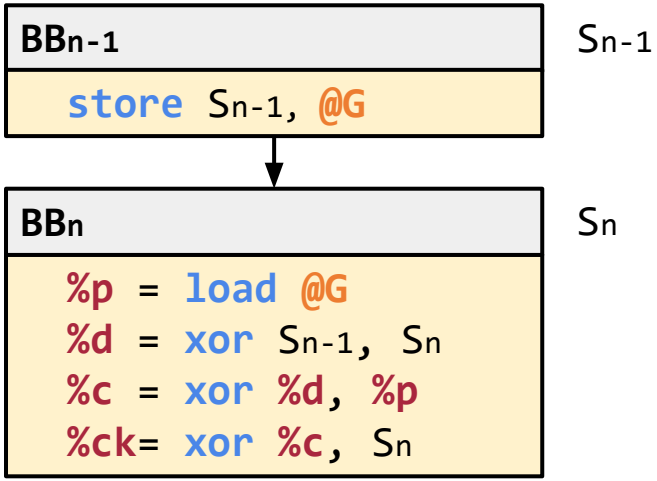
Entire datapath protection along with low runtime overhead.

ConDa: Lazy Checking Motion

- Using **instruction duplication** to detect data-flow errors.
- Using **software signatures** to detect control-flow errors.
- **Key design**: **Buffering** all checking results for *one basic block* into *a single register*.

```
%A  = load %addr1
%A' = load %addr1
%B  = mul  %A , 3
%B' = mul  %A', 3
%C  = add %A , %B
%C' = add %A', %B'
%ck= icmp %C, %C'
```

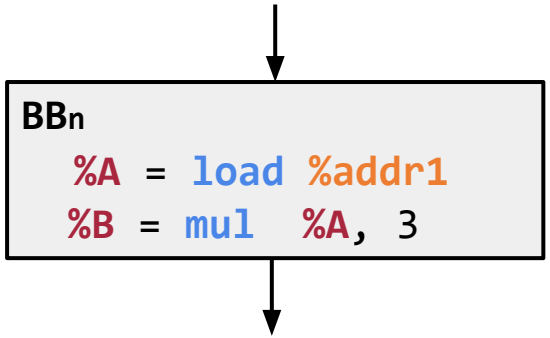
Code with instruction duplication

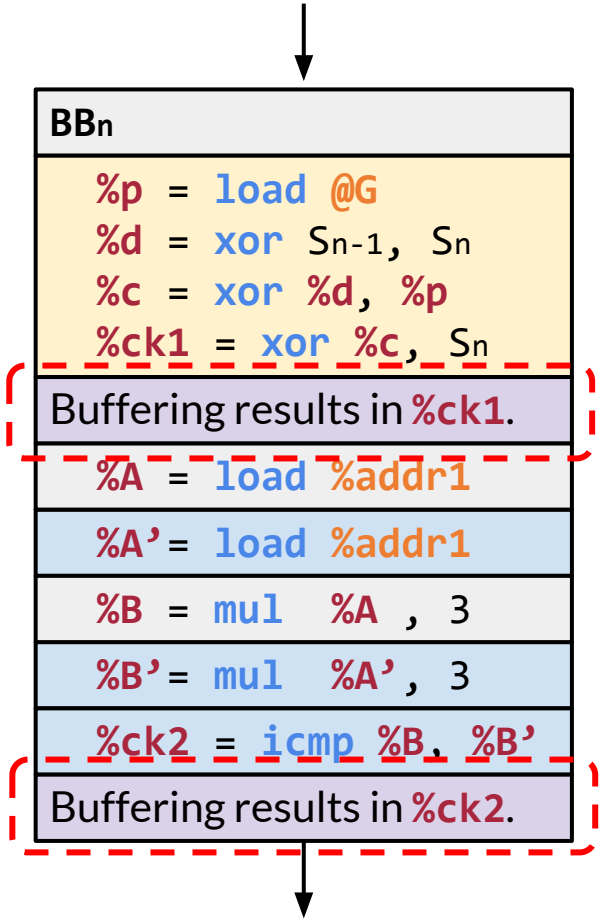


Code with software signature

ConDa: Lazy Checking Motion

- A code example of lazy checking motion.

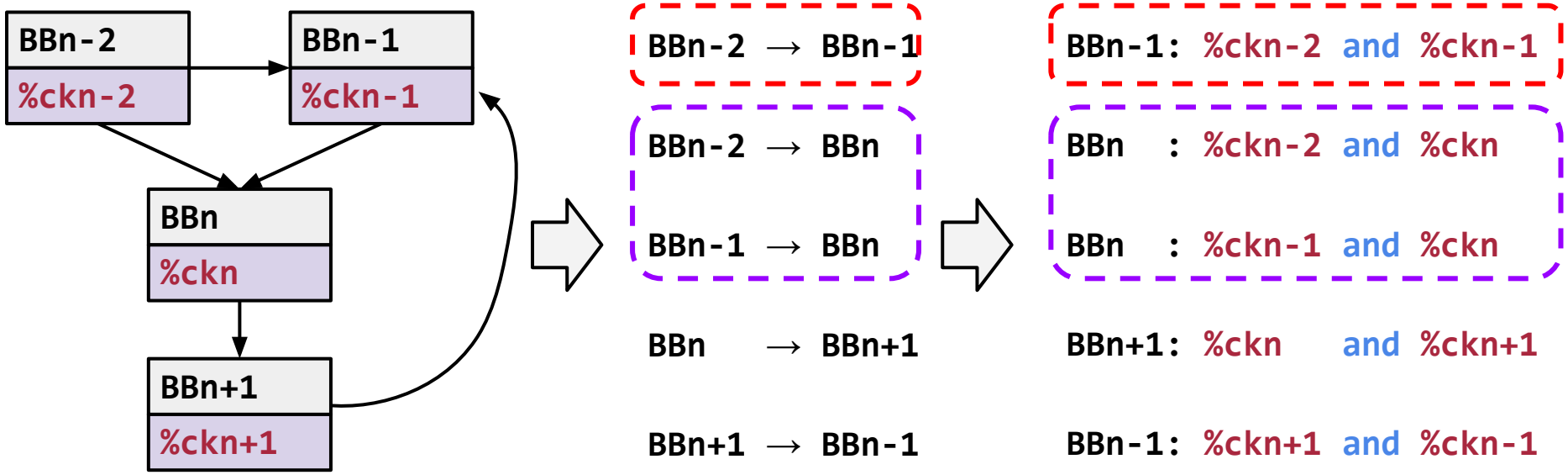




Preserving the program control-flow while still ensuring protection effectiveness.

ConDa: Domination Propagation

- **Key design**: while buffering checking results across basic blocks, using only **register-level** computation by performing a domination analysis.



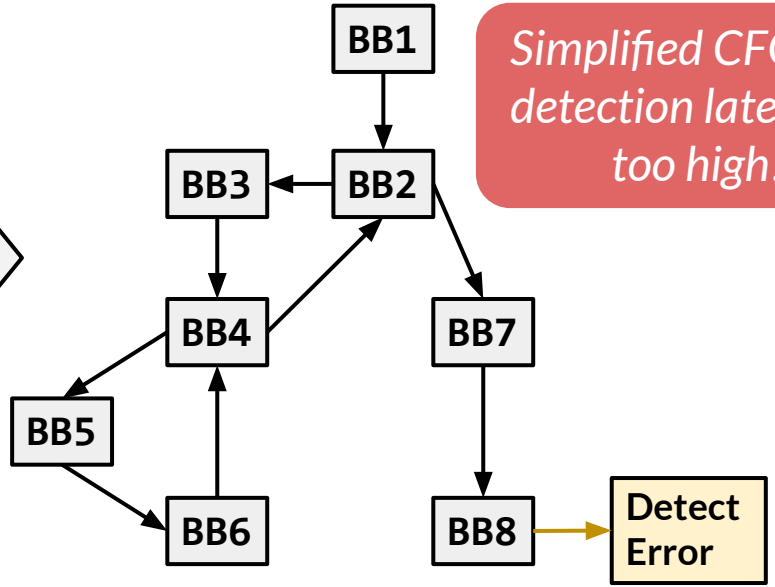
Inside a function, each basic block buffers all checking results of its predecessors in runtime.

ConDa: Adaptive Detection Placement

- We have buffered error-checking results *inside & across* basic blocks in each function, but where to insert **error-detection branches**?

```
int HPC_sparsemv(HPC_Sparse_Matrix *A,
                 double * const x,
                 double * const y)
{
      ...
      for (int i=0; i<nrow; i++)
      {
            ...
            for (int j=0; j<cur_nnz; j++)
            ...
            ...
      }
      ...
      return(0);
}
```

Program in C code

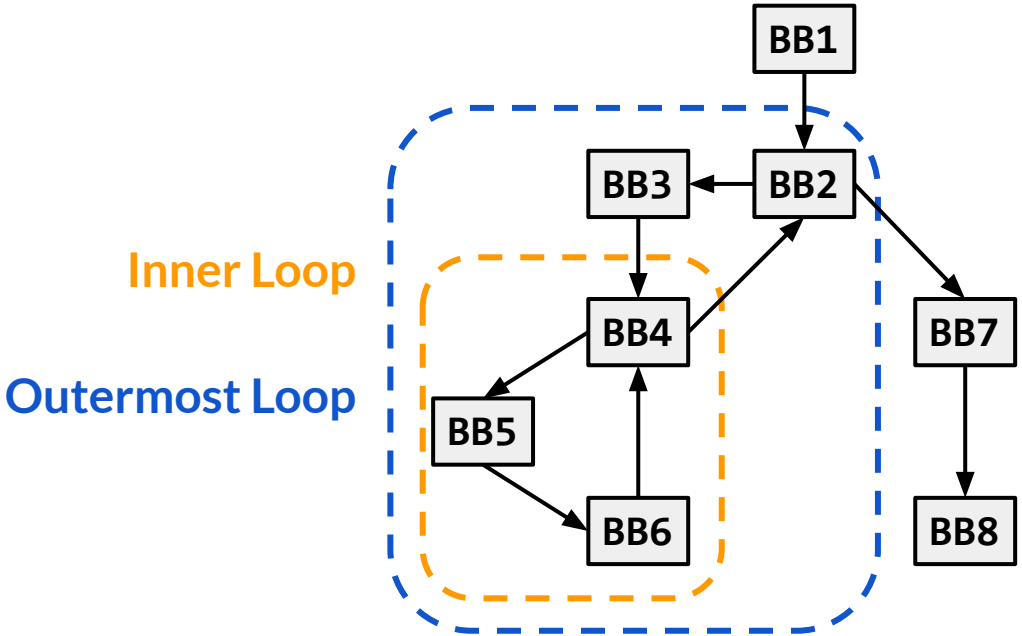


Simplified CFG, but detection latency is too high!

Program in CFG

ConDa: Adaptive Detection Placement

- **Key design**: Besides the branch at function exit, **placing error-detection branches** based on **number of outermost loops** within this function.

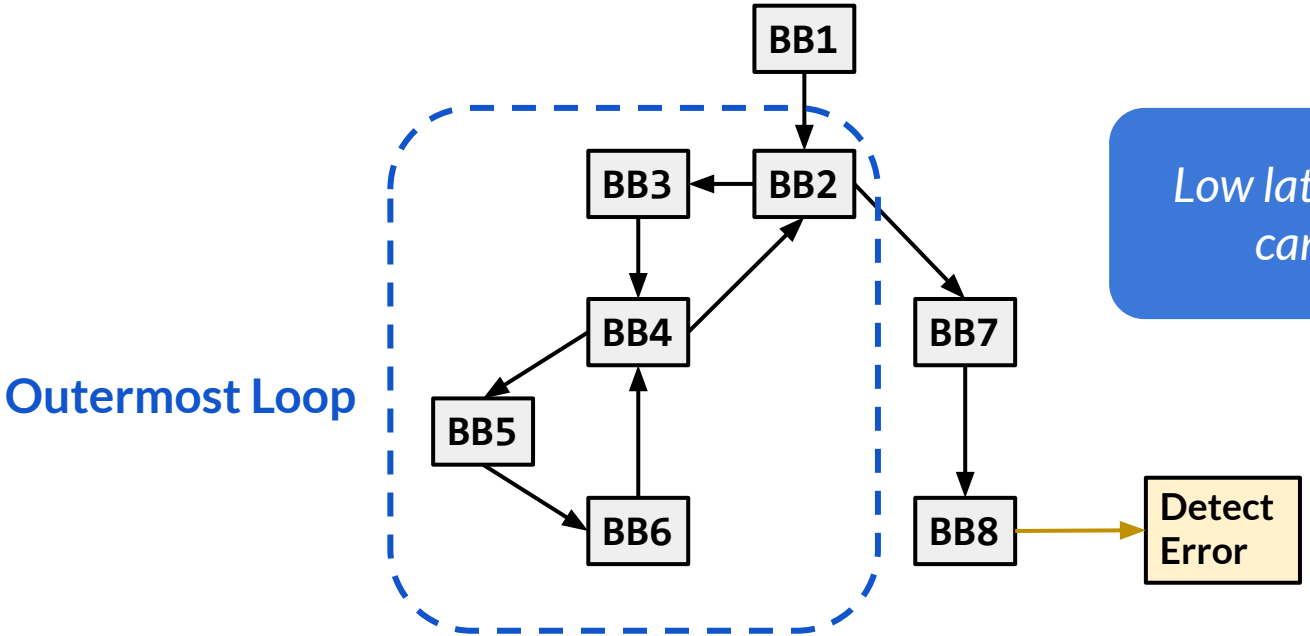


The more outermost loops, the more complex CFG.

Reducing detection latency with a relatively simplified CFG.

ConDa: Adaptive Detection Placement

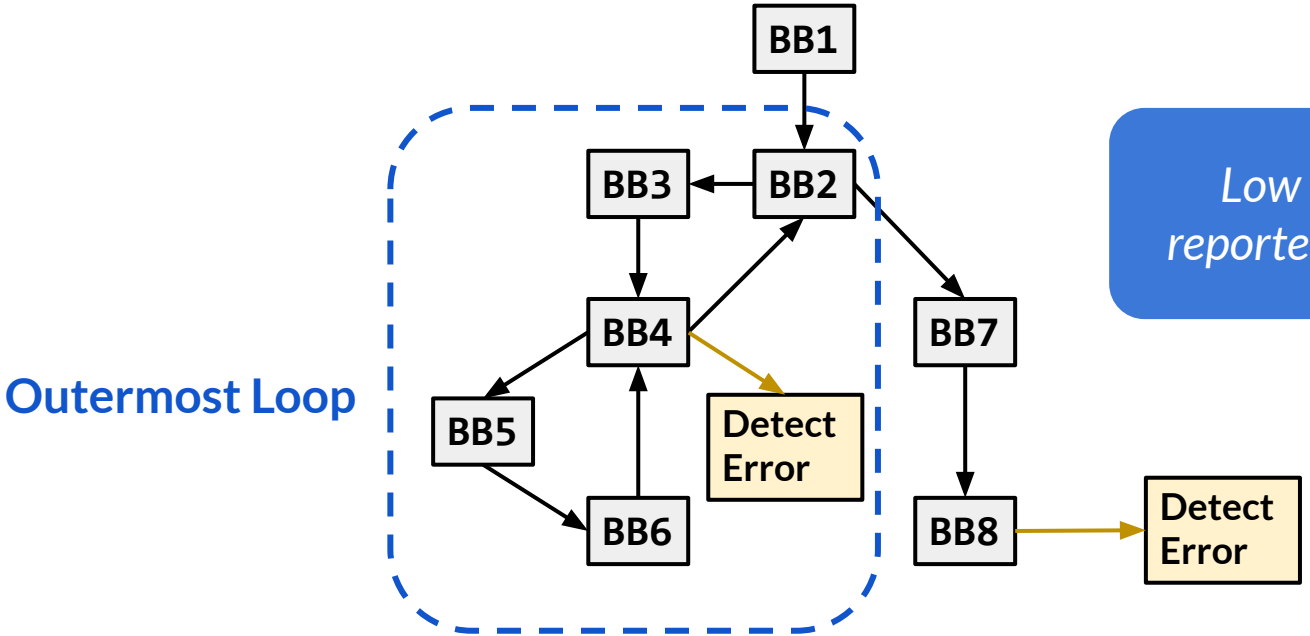
- Number of outmost loop is **0**.
- Very simple function, only placing one at **function exit**.



Low latency. No-loop function can execute very fast.

ConDa: Adaptive Detection Placement

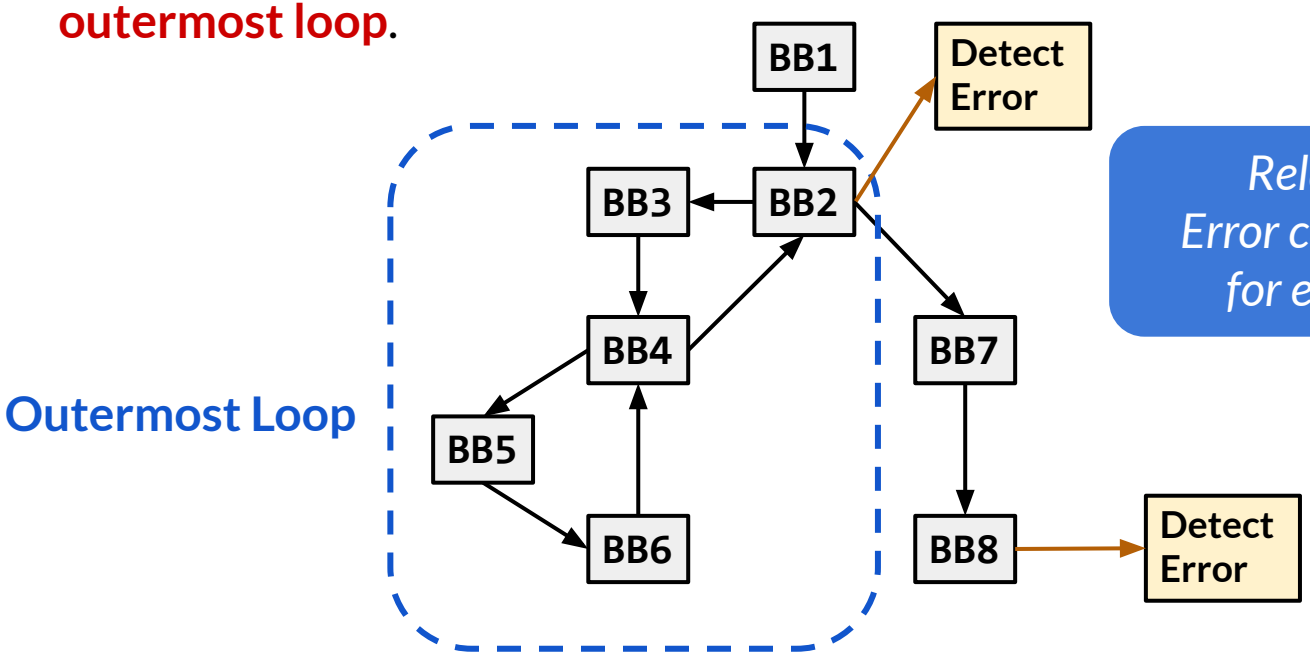
- Number of outmost loop is **1**.
- Not that complex, besides function exit one, also placing one **inside this loop**.



Low latency. Error can be reported within each iteration.

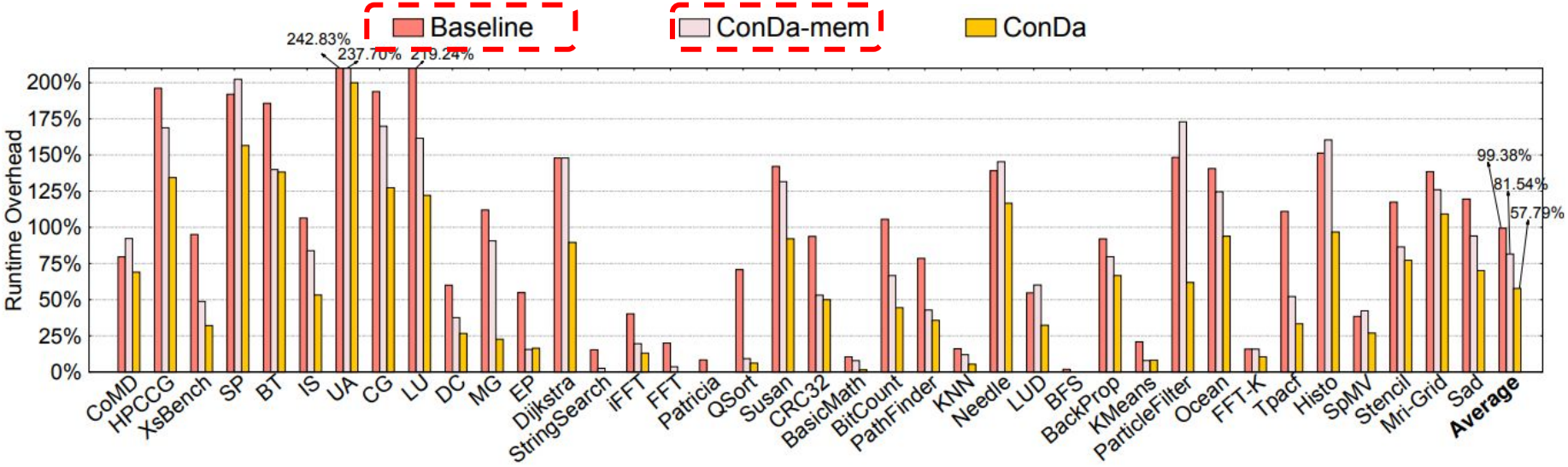
ConDa: Adaptive Detection Placement

- Number of outmost loop is **greater than 1**.
- Likely to be complex, besides function exit one, placing branch **outside each outermost loop**.



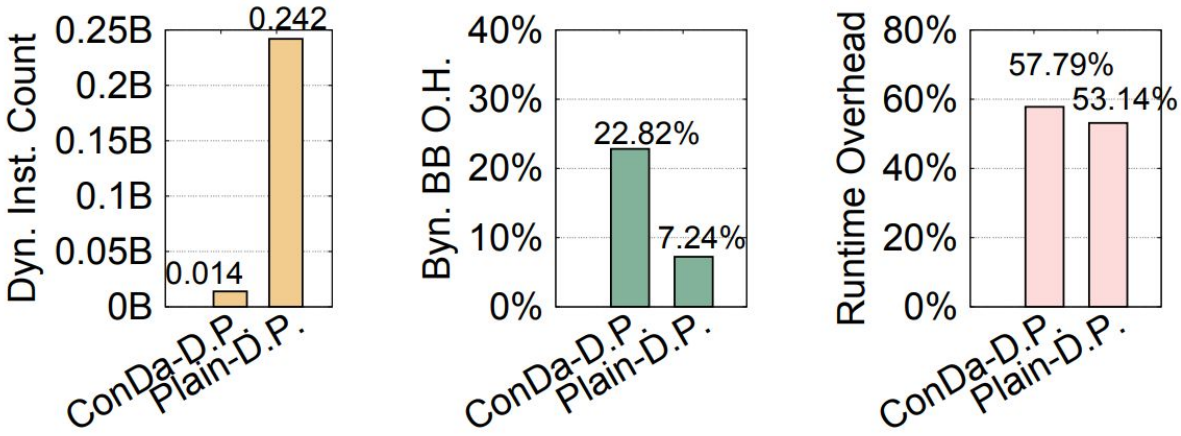
Relatively low latency.
Error can at least be reported for each outermost loop.

Evaluation: Runtime Overhead



- On average of 38 benchmarks, ConDa has only **57.79%** runtime overhead.
- **~40%** faster than baseline methods with **the same fault coverage**.
- ConDa is **~30%** faster than ConDa-mem, demonstrating Domination Propagation.

Evaluation: Detection Latency



Detection Latency | Control-flow Complexity | Runtime Overhead

- Plain Detection Placement (D.P.) indicates “only placing branch at function exit”.
- Reducing **16.28×** detection latency using Adaptive Detection Placement.
- Preserved control-flow and runtime overhead compared with Plain D.P.

Summary

- Existing datapath protection has excessive runtime overhead due to complex program control-flow and frequent memory accesses.
- ConDa is an entire datapath soft error detection technique with low overhead.
- ConDa consists of one static LLVM IR-level code transformation.
- By maintaining control-flow, using register-level updates, and adaptively placing detection branches, ConDa has 57.79% overhead with low detection latency.

Yafan Huang
University of Iowa
yafan-huang@uiowa.edu
https://hyfshishen.github.io